# Optimal logarithmic-degree depth for monotone algebraic formulas

Hervé Fournier, Nutan Limaye, Guillaume Malod, Srikanth Srinivasan, Sébastien Tavenas

doi:10.4230/LIPIcs.CCC.2023.28; CC BY4.0

*Editorial note.* Original authors: Hervé Fournier, Nutan Limaye, Guillaume Malod, Srikanth Srinivasan, Sébastien Tavenas. arXiv:2302.06984v1 (2023-02-14), https://arxiv.org/abs/2302.06984v1. Creative Commons Attribution4.0, https://creativecommons.org/licenses/by/4.0/.

*Editorial note.* Selected complete author native main statements/proofs, necessary context, original TikZ code and complete bibliography in standalone layout. Unused frontmatter/packages omitted; no mathematical changes or new proofs. Original source is CC BY4.0.

*Allowed standard prerequisites (editorial).*

- Standard formula depth reduction and Raz homogenization. Exact prior theorem statements in author preliminaries retained; near-linear Bonet-Buss proof also retained where used.

**Theorem 1** (Lower Bound)**.** *Let $n$ and $d = d(n)$ be growing parameters such that $d(n) \leq \sqrt{n}$. Then there is a monotone algebraic formula $F$ of size at most $n$ and depth $O(\log d)$ computing a polynomial $P \in \mathbb{F}[x_1, \ldots, x_n]$ of degree at most $d$ such that any monotone formula $F'$ of depth $o(\log d)$ computing $P$ must have size $n^{\omega(1)}$.*

# 1 Preliminaries

**Basic notation.** Throughout, unless otherwise specified, we work with polynomials over a field $\mathbb{F}$. We will work with the multivariate ring of polynomials $\mathbb{F}[x_1, \ldots, x_n]$ or its *non-commutative* analog $\mathbb{F}\langle x_1, \ldots, x_n\rangle$.

## 1.1 Algebraic formulas

We start with some brief definitions and results related to algebraic formulas. For much more about this model, see the standard references [SY10, Sap15].

**The model.** An algebraic formula over the multivariate polynomial ring $\mathbb{F}[x_1, \ldots, x_n]$ is a rooted, directed tree with edges directed towards the root. Leaves are labelled by variables $x_1, \ldots, x_n$ or by the constant 1 and edges by non-zero field

constants. Internal nodes (i.e., gates) by $+$ and $\times$ and compute linear combinations (based on the edge weights) or products of their children. We will assume, with loss of generality, that if a node $\alpha$ has for child a leaf labelled by 1, then $\alpha$ is a $+$-gate and that if a $+$-gate $\alpha$ has only children labelled by 1, then $\alpha$ is the output of the formula.[1] A *non-commutative* algebraic formula over the multivariate polynomial ring $\mathbb{F}\langle x_1, \ldots, x_n \rangle$ is defined similarly, with the additional assumption that the children of any $\times$-gate are linearly ordered, and the corresponding product is computed in this order.

Unless explicitly stated, the algebraic formulas we consider have *unbounded* fan-in (i.e., a gate can have any number of inputs). The *size* of $F$ will denote the number of leaves,[2] the *depth* of $F$ the longest leaf-to-root path. The *product-depth* and the *sum-depth* of $F$ are defined to be the maximum number of product gates and sum gates encountered on a leaf-to-root path, respectively.

A *parse tree* of a formula $F$ is a subformula of $F$ which corresponds to the way a monomial is built in the evaluation of $F$. Parse trees of $F$ can be defined inductively as follows:

- If $F$ has a top $+$-gate, a parse tree of $F$ is obtained by taking a parse tree of one of its children together with the corresponding edge to the root of $F$;
- If $F$ has a top $\times$-gate, a parse tree of $F$ is obtained by a taking a parse tree of each of its children, together with the incoming edges of the root of $F$;
- The only parse tree of a leaf is itself.

The polynomial computed by a parse tree is a single monomial, which is equal to the product of the variables labelling its leaves, multiplied by the product of the scalars labelling its edges. The polynomial computed by a formula $F$ is easily seen to be the sum of the monomials computed by all its parse trees.

A formula $F$ is called *monotone* if any monomial computed by a parse tree of $F$ has a non-zero coefficient in the polynomial computed by $F$.

We now recall some well-known results from the literature regarding algebraic formulas. It should be noted that these results (specifically Theorems 2, 3 and 4 later on) are usually proved in the general setting of commutative formulas. However, it is easy to see that the proofs of these results carry over to the monotone, non-commutative setting without significant change.

**Depth-reduction.** Classical results [BKM73, Bre74] show that any algebraic formula of small size can be simulated by one of small depth and not much larger size. Formally,

**Theorem 2.** *Let $F$ be a (non-commutative or commutative) algebraic formula of size $s$ computing a polynomial $P$. Then there is an algebraic formula $F'$ of size at most* $\mathrm{poly}(s)$ *and depth $\Delta = O(\log s)$ computing $P$. We may also assume that each gate in $F'$ has fan-in 2. Furthermore, $F$ is homogeneous and/or monotone, then so is $F'$.*

[1] This ensures that a formula can compute polynomials with a constant term but forbids using many arithmetic operations just to compute constants.

[2] This is within a constant factor of the number of gates, as long as each gate has fan-in at least 2 each (which is without loss of generality).

**Homogeneity.** Each gate in an algebraic formula has a *syntactic degree* defined in a natural way. Leaves labelled by the constant 1 have syntactic degree 0, leaves labelled with a variable have syntactic degree 1, $\times$-gates have a syntactic degree that is the sum of the syntactic degrees of their children, and $+$-gates have a syntactic degree that is equal to the largest of the syntactic degrees of their children. The syntactic degree of a formula is defined as the syntactic degree of its output. Notice that in a formula the syntactic degree of any gate is bounded by the syntactic degree of the formula.

We will further assume that no gate computes the zero polynomial.

A formula is *homogeneous* if each gate in the formula computes a homogeneous polynomial. Equivalently, in terms of syntactic degrees, this means that all the children of a sum gate have the same syntactic degree.

Raz [Raz13] showed how to convert a possibly inhomogeneous formula $F$ to a homogeneous formula with a relatively small blow-up in size.

**Theorem 3.** *Let $F$ be a (non-commutative or commutative) algebraic formula of size $s$ and product-depth $\Delta$ computing a polynomial $P$ such that all gates in $F$ have fan-in 2. Then there is a* homogeneous *algebraic formula $F'$ of size at most $O\left(s \cdot \binom{\Delta+d+1}{d}\right)$ and product-depth $\Delta$ computing $P$. In particular, if $\Delta = O(\log s)$ and $d = O(\log s)$, then the formula $F'$ has size* $\mathrm{poly}(s)$.

**Theorem 4** (Depth-reduction with near-linear size)**.** *The following holds for any $\varepsilon > 0$. Let $F$ be a (non-commutative or commutative) algebraic formula of size $s$ computing a polynomial $P$. Then there is an algebraic formula $F'$ of size at most $s^{1+\varepsilon}$ and depth $\Delta = 2^{O(1/\varepsilon)} \cdot \log s$ computing $P$. We may also assume that each gate in $F'$ has fan-in 2. Furthermore, if $F$ is homogeneous and/or monotone, then so is $F'$.*

**Theorem 5.** *Let $n$ and $d = d(n)$ be growing parameters such that $d(n) \leq \sqrt{n}$. Then there is a monotone algebraic formula $F$ of size at most $n$ and depth $O(\log d)$ computing a polynomial $P \in \mathbb{F}[x_1, \ldots, x_n]$ of degree at most $d$ such that any monotone formula $F'$ of depth $o(\log d)$ computing $P$ must have size $n^{\omega(1)}$.*

# 2 Tightness

Given integers $k \geq 1$ and $r \geq 2$, we will define a polynomial $H^{(k,r)}$. Intuitively, we want to define this polynomial as a standard universal polynomial for formulas. It is composed of $k$-nested inner products, each one of size $r$. In the following we will drop the superscript in $H^{(k,r)}$ and write simply $H$ instead.

The polynomial $H$ will be defined over the set of $(2r)^k$ variables

$$\{x_{\sigma,\tau} \mid \sigma \in [2]^k, \tau \in [r]^k\}.$$

Let us define recursively polynomials $H_{u,v}$ for all $(u, v) \in [2]^{\leq k} \times [r]^{\leq k}$ such that $|u| = |v|$:

$$\begin{aligned} H_{u,v} &= x_{u,v} && \text{when } |u| = |v| = k \\ H_{u,v} &= \sum_{a=1}^{r} H_{u1,va} H_{u2,va} && \text{otherwise.} \end{aligned}$$

The polynomial $H$ is defined as the polynomial $H_{\varepsilon,\varepsilon}$. Note that $H$ is a polynomial of degree $d = 2^k$ and has $r^{d-1}$ monomials.

From its definition, $H$ is computed by a monotone formula $M$ of size $(2r)^k$ and depth $2k$, with a $+$-gate at the top, alternating layers of $+$-gates and $\times$-gates, with $+$-gates of fan-in $r$ and $\times$-gates of fan-in 2, and leaves labelled with distinct variables.

For words $u$ and $v$ over the same alphabet, we write $u \sqsupset v$ if $u$ is a prefix of $v$. There is a natural one-to-one correspondance between prefixes of words of $([r] \times [2])^k$ and nodes of $M$, which is the following. Let $\sigma = \sigma_1 \dots \sigma_k \in [2]^k$, and $\tau = \tau_1 \dots \tau_k \in [r]^k$. The word $\tau_1\sigma_1 \dots \tau_k\sigma_k$ corresponds to a path from the root of $M$ to the leaf labelled $x_{\sigma,\tau}$, while proper prefixes of $\tau_1\sigma_1 \dots \tau_k\sigma_k$ correspond to internal gates in $M$ along this path. For $\ell < k$, $u = u_1 \dots u_\ell \in [2]^\ell$ and $v = v_1 \dots v_\ell \in [r]^\ell$, the node which corresponds to the word $v_1u_1 \dots v_\ell u_\ell$ is the $+$-gate of $M$ computing $H_{u,v}$.

The polynomial $H$ is easily seen to be set-multilinear with respect to the sets of variables $\{X_\sigma \mid \sigma \in [2]^k\}$ where $X_\sigma = \{x_{\sigma,\tau} \mid \tau \in [r]^k\}$. This means that each monomial has exactly one variable from each set $X_\sigma$.

**Remark 6.** *For $|u| = |v| = \ell \leq k$, $u \in [2]^\ell$ and $v \in [r]^\ell$, the polynomial $H^{(k,r)}_{u,v}$ is defined over the set of variables*

$$X_{u,v} = \{x_{\sigma,\tau} \mid \sigma \in [2]^k,\ \tau \in [r]^k, u \sqsupset \sigma,\ v \sqsupset \tau\}$$

*and is the polynomial $H^{(k-\ell,r)}$ (upto renaming of the variables). In particular, its degree is $d' = 2^{k-\ell}$, it has $r^{d'-1}$ monomials and it is set-multilinear with respect to $\{X_\sigma \mid \sigma \in [2]^k,\ u \sqsupset \sigma\}$.*

Before proving hardness of $H$ for small-depth monotone formulas, we need to show that the gates of a monotone formula computing $H$ cannot compute too many monomials. This is proved in Lemma 8 below.

**Proposition 7.** *Consider two variables $x_{\sigma,\tau}$ and $x_{\sigma',\tau'}$. If $x_{\sigma,\tau}x_{\sigma',\tau'}$ appears in a monomial of $H$, and if $\sigma$ and $\sigma'$ have a common prefix of length $\ell < k$, then $\tau$ and $\tau'$ have a common prefix of length $\ell + 1$.*

*Proof.* Observe that $x_{\sigma_1,\tau_1} \dots x_{\sigma_p,\tau_p}$ is a monomial of $H$ if and only if these variables form the leaves of a parse tree of $M$. As observed above, the root-to-leaf path leading to the variable $x_{\sigma_i,\tau_i}$ in $M$ is obtained by taking in turn the first letter of $\tau_i$, the first letter of $\sigma_i$, the second letter of $\tau_i$, etc.

If the product $x_{\sigma,\tau}x_{\sigma',\tau'}$ appears in a monomial of $H$, it must be possible to complete the union of the two paths, from the root to $x_{\sigma,\tau}$ and from the root to $x_{\sigma',\tau'}$, into a parse tree of the formula $M$.

If the longest common prefix of $\tau$ and $\tau'$ were of length at most $\ell$, then the lowest common ancestor of $x_{\sigma,\tau}$ and $x_{\sigma',\tau'}$ in $M$ would be a $+$-gate, which is not possible in a parse tree where each $+$-gate has a single child. □

**Lemma 8.** *If $F$ is a monotone formula which computes the polynomial $H$ and if $\alpha$ is a gate of $F$ of degree $d_\alpha$, then the number of monomials of the polynomial computed at gate $\alpha$ is at most $r^{d_\alpha-1}$.*

*Proof.* Since $H$ has $r^{d-1}$ monomials, the result is true when $d_\alpha = d$ by monotonicity. Assume now that $d_\alpha < d$.

Let

$$I = \{\sigma \in [2]^k \mid \text{some variable of } X_\sigma \text{ appears in } \alpha\}.$$

For $u \in [2]^{\leqslant k}$, let $I_u = \{\sigma \in [2]^k \mid u \sqsupset \sigma\}$. Let $\{u_1, \dots, u_p\}$ be the set of words $w$ of minimal length such that $I_w \subseteq I$. Then $I$ is the disjoint union $\bigcup_{\ell \in [p]} I_{u_\ell}$. Since $d_\alpha < d$, $I \neq [2]^k$ so no $u_\ell$ is the empty word.

Consider some $\ell \in [p]$. Let $\bar{u}_\ell$ be obtained by switching the last letter of $u_\ell$. By minimality of the length of $u_\ell$, we must have $I_{\bar{u}_\ell} \not\subseteq I$. Let $\sigma$ be a word in $I_{\bar{u}_\ell} \setminus I$. Since $F$ is monotone and computes a set-multilinear polynomial, it is a set-multilinear formula and therefore the polynomial computed at $\alpha$ must be multiplied by some variable $x_{\sigma,\tau}$. Let $v_\ell$ be the prefix of length $|u_\ell|$ of $\tau$. Consider any variable $x_{\sigma',\tau'}$ appearing in $\alpha$ such that $u_\ell \sqsupset \sigma'$. Since the product $x_{\sigma,\tau}x_{\sigma',\tau'}$ must appear in a monomial of $H$ by monotonicity, it must be that $v_\ell \sqsupset \tau'$ by Proposition 7.

Any monomial $m$ in the polynomial computed in gate $\alpha$ can be written in a unique way $m = m_1 \cdots m_p$ with $m_\ell$ set-multilinear with respect to $\{X_\sigma \mid u_\ell \sqsupset \sigma\}$. By the above, $m_\ell$ is a monomial over the variables $X_{u_\ell,v_\ell}$ of degree $|I_{u_\ell}|$. By monotonicity, it should be possible to complete leaves of $M$ labelled with variables from $m_\ell$ into a parse tree of $M$ appearing in $H$, which proves that $m_\ell$ is a monomial of $H_{u_\ell,v_\ell}$. There are at most $r^{|I_{u_\ell}|-1}$ such submonomials $m_\ell$ by Remark 6. It follows that the number of monomials of the polynomial computed in node $\alpha$ is at most

$$\prod_{\ell=1}^{p} r^{|I_{u_\ell}|-1} \leq r^{|I|-1} = r^{d_\alpha - 1}. \qquad \square$$

We are ready to prove hardness of the polynomial $H$ for monotone computation. We shall make use of the following "product lemma", which comes in different forms in e.g. [SY10, HY11, Sap15].

**Lemma 9.** *A degree-$d$ homogeneous formula $F$ of size $s$ and product-depth $\Delta$ can be written as a sum of $O(s)$ polynomials, each of which is a product of $\Omega(\Delta d^{1/\Delta})$ many polynomials of positive degree. Moreover, each of these polynomials is computed by some gate in $F$.*

**Proposition 10.** *If $F$ is a monotone formula of product-depth $\Delta \leq \log d$ which computes $H$, then its size is at least $r^{\Omega(\Delta d^{1/\Delta})}$.*

*Proof.* Let $t = \Delta d^{1/\Delta}$. Since $F$ is monotone, it is homogeneous and by Lemma 9 can be written as

$$F = \sum_{i=1}^{s'} \prod_{j=1}^{t_i} F_{i,j}$$

with $s' = O(s)$ and $t_i = \Omega(t)$ for all $i$, and $F_{i,j}$ is of degree at least 1 and computed by some gate in $F$.

Let $d_{i,j}$ be the degree of $F_{i,j}$. By Lemma 8, each $F_{i,j}$ computes at most $r^{d_{i,j}-1}$ monomials. The number of monomials of $\prod_{i=1}^{t_j} F_{i,j}$ is therefore bounded by

$$\prod_{j=1}^{t_i} r^{d_{i,j}-1} \leq r^{\sum_{j=1}^{t_i} d_{i,j} - t_i} \leq r^{d-t}$$

since $\sum_{j=1}^{t_i} d_{i,j} = d$. It follows that the number of monomials computed by $F$ is at most $s \cdot r^{d-t}$. Since $F$ computes $H$ which has $r^{d-1}$ monomials, we get $s = r^{\Omega(t)}$. □

We can now get Theorem 1 from the introduction, which is restated here for convenience.

*Proof of Theorem 1.* Choose parameters $k(n)$ and $r(n)$ such that $P(n) := H^{(k,r)}$ has $\Theta(n)$ variables and degree $\Theta(d)$: let $k = \log d$ and $r = \frac{1}{2}n^{1/\log d}$. Condition $d(n) \leq \sqrt{n}$ ensures that $r \geqslant 2$. The polynomial $P$ has a monotone formula of size $O(n)$ and depth $O(\log d)$. By Proposition 10, any monotone formula of product-depth $\Delta \leq \log d$ computing $P$ has size

$$r^{\Omega(\Delta d^{1/\Delta})} = \left(\frac{1}{2}n^{1/\log d}\right)^{\Omega(\Delta d^{1/\Delta})} = (n/d)^{\left(\frac{\Delta d^{1/\Delta}}{\log d}\right)}$$

which is $n^{\Omega\left(\frac{\Delta d^{1/\Delta}}{\log d}\right)}$ using the hypothesis $d(n) \leq \sqrt{n}$. Since $\frac{\Delta d^{1/\Delta}}{\log d} \to +\infty$ when $\Delta = o(\log d)$ this bound is $n^{\omega(1)}$. □

# References


[AV08] Manindra Agrawal and V. Vinay. Arithmetic circuits: A chasm at depth four. In *proceedings of Foundations of Computer Science (FOCS)*, pages 67–75, 2008.

[B00] Peter Bürgisser. Cook's versus Valiant's hypothesis. *Theoretical Computer Science*, 235(1):71–88, 2000.

[BB94] Maria Luisa Bonet and Samuel R. Buss. Size-depth tradeoffs for boolean formulae. *Information Processing Letters*, 49(3):151 – 155, 1994.

[BCE95] Nader H. Bshouty, Richard Cleve, and Wayne Eberly. Size-depth tradeoffs for algebraic formulas. *SIAM J. Comput.*, 24(4):682–705, 1995.

[BKM73] R. Brent, D. Kuck, and K. Maruyama. The parallel evaluation of arithmetic expressions without division. *IEEE Transactions on Computers*, C-22(5):532–534, 1973.

[Bre74] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, April 1974.

[CKSV16] Suryajith Chillara, Mrinal Kumar, Ramprasad Saptharishi, and V. Vinay. The chasm at depth four, and tensor rank : Old results, new insights. *CoRR*, abs/1606.04200, 2016.

[CLS18] Suryajith Chillara, Nutan Limaye, and Srikanth Srinivasan. Small-depth multilinear formula lower bounds for iterated matrix multiplication, with applications. In *STACS*, volume 96 of *LIPIcs*, pages 21:1–21:15. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2018.

[GKKS16] Ankit Gupta, Pritish Kamath, Neeraj Kayal, and Ramprasad Saptharishi. Arithmetic circuits: A chasm at depth 3. *SIAM Journal of Computing*, 45(3):1064–1079, 2016.

[HY11] Pavel Hrubeš and Amir Yehudayoff. Homogeneous formulas and symmetric polynomials. *Comput. Complexity*, 20(3):559–578, 2011.

[KdOS19] Mrinal Kumar, Rafael Mendes de Oliveira, and Ramprasad Saptharishi. Towards optimal depth reductions for syntactically multilinear circuits. In Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, editors, *46th International Colloquium on Automata, Languages, and Programming, ICALP 2019, July 9-12, 2019, Patras, Greece*, volume 132 of *LIPIcs*, pages 78:1–78:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.

[Koi12] Pascal Koiran. Arithmetic circuits: The chasm at depth four gets wider. *Theor. Comput. Sci.*, 448:56–65, 2012.

[Kos86] S Rao Kosaraju. Parallel evaluation of division-free arithmetic equations. In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, pages 231–239, 1986.

[KSS14] Neeraj Kayal, Chandan Saha, and Ramprasad Saptharishi. A super-polynomial lower bound for regular arithmetic formulas. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 146–153. ACM, 2014.

[LST21a] Nutan Limaye, Srikanth Srinivasan, and Sébastien Tavenas. Super-polynomial lower bounds against low-depth algebraic circuits. *Electron. Colloquium Comput. Complex.*, TR21-081, 2021.

[LST21b] Nutan Limaye, Srikanth Srinivasan, and Sébastien Tavenas. Super-polynomial lower bounds against low-depth algebraic circuits. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, pages 804–814. IEEE, 2021.

[Nis91] Noam Nisan. Lower bounds for non-commutative computation. In *Proceedings of the twenty-third annual ACM symposium on Theory of computing*, pages 410–418, 1991.

[NW97] Noam Nisan and Avi Wigderson. Lower bounds on arithmetic circuits via partial derivatives. *Computational Complexity*, 6(3):217–234, 1997.

[PM75] Franco P. Preparata and David E. Muller. The time required to evaluate division-free arithmetic expressions. *Inf. Process. Lett.*, 3(5):144–146, 1975.

[PM76] Franco P. Preparata and David E. Muller. Efficient parallel evaluation of boolean expression. *IEEE Trans. Computers*, 25(5):548–549, 1976.

[Raz06] Ran Raz. Separation of multilinear circuit and formula size. *Theory of Computing*, 2(1):121–135, 2006.

[Raz13] Ran Raz. Tensor-rank and lower bounds for arithmetic formulas. *Journal of the ACM*, 60(6):40:1–40:15, 2013.

[RY08] Ran Raz and Amir Yehudayoff. Balancing syntactically multilinear arithmetic circuits. *Computational Complexity*, 17(4):515–535, 2008.

[Sap15] Ramprasad Saptharishi. A survey of lower bounds in arithmetic circuit complexity. Github survey, 2015.

[Spi71] Philip M. Spira. On time hardware complexity tradeoffs for Boolean functions. In Shu Lin, editor, *Proceedings of the Fourth Hawaii International Conference on System Sciences*, pages 525–527. Western Periodicals Company, North Hollywood, California, 1971.

[SS80] Eli Shamir and Marc Snir. On the depth complexity of formulas. *Math. Syst. Theory*, 13:301–322, 1980.

[SW01] Amir Shpilka and Avi Wigderson. Depth-3 arithmetic circuits over fields of characteristic zero. *Computational Complexity*, 10(1):1–27, 2001.

[SY10] Amir Shpilka and Amir Yehudayoff. Arithmetic circuits: A survey of recent results and open questions. *Foundations and Trends in Theoretical Computer Science*, 5:207–388, March 2010.

[Tav15] Sébastien Tavenas. Improved bounds for reduction to depth 4 and depth 3. *Information and Computation*, 240:2–11, 2015.

[TLS22] Sébastien Tavenas, Nutan Limaye, and Srikanth Srinivasan. Set-multilinear and non-commutative formula lower bounds for iterated matrix multiplication. In *STOC*, pages 416–425. ACM, 2022.

[VSBR83] Leslie G. Valiant, Sven Skyum, Stuart J. Berkowitz, and Charles Rackoff. Fast Parallel Computation of Polynomials Using Few Processors. *SIAM Journal of Computing*, 12(4):641–644, 1983.

[YK68] SV Yablonskii and VP Kozyrev. Mathematical problems of cybernetics. *Information Materials of Scientific Council of Akad. Nauk SSSR on Complex Problem "Kibernetika*, 19:3–15, 1968.